Ethan Chung

ICS 661

Advanced AI

Fall 2024

Section 1: Task Description

The goal of this project is to create a simple Multilayer Perceptron (MLP) model using the provided training set and evaluate its performance with the testing set to see how the backpropagation training algorithm performs.

Section 2: Model Description

This model is constructed with an input layer of 784 input nodes, 3 hidden layers of size 50, and an output layer of size 10, representing the possible labels 0-9. The hidden layer size was chosen through hyper-parameter tuning through GridSearchCV.

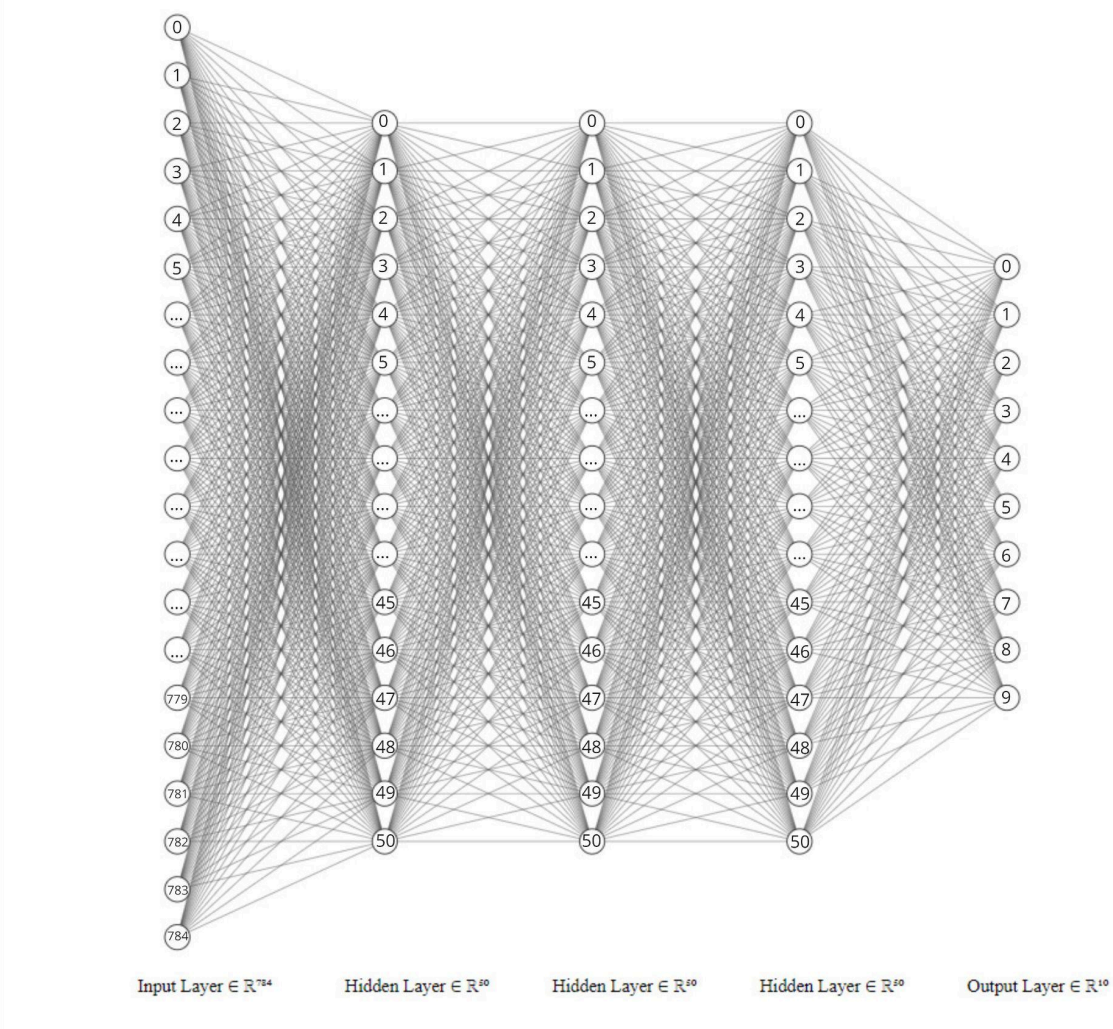

Figure 1: A FCNN style diagram, showing the structure of the neural network.

Section 3: Experiment Settings

3.1 Dataset Description

No specific information is known about the dataset. It is provided as two files, pre-split between training and testing sets, with 60,000 and 10,000 data samples respectively. There are 785 columns in both sets, with the first column representing a label between 0-9, and the other 784 columns representing features that contain numbers between 0-255 (potentially representing some color value).

3.2 Detailed Experimental Setups

The training dataset is given as 60,000 rows, with 784 feature columns and 1 label column. The label column is dropped so that only features are trained. The test dataset is given as 10,000 rows, with the same number of columns. The first column, the label column, is dropped during training.

Model settings were hyperparameter-tuned with GridSearchCV, fit on the training set and validated through the testing set, searching through the following parameter space:

```
parameter_space = {
    'hidden_layer_sizes': [(50,50,50), (50,100,50), (100,)],
    'activation': ['tanh', 'relu'],
    'solver': ['sgd', 'adam'],
    'alpha': [0.0001, 0.05],
    'learning_rate': ['constant','adaptive'],
}
```

After performing hyperparameter tuning, the best parameters from the search space was found to be:

{'activation': 'relu', 'alpha': 0.0001, 'hidden_layer_sizes': (50, 50, 50), 'learning_rate': 'constant', 'solver': 'adam'}

A classification report was generated on the test set after the completion of the hyperparameter tuning

Results on the test set:

	precision	recall	f1-score	support
0	0.97	0.97	0.97	980
1	1.00	0.97	0.98	1135
2	0.92	0.95	0.94	1032
3	0.91	0.95	0.93	1010
4	0.92	0.94	0.93	982
5	0.95	0.93	0.94	892
6	0.95	0.96	0.95	958
7	0.96	0.94	0.95	1027
8	0.95	0.91	0.93	974
9	0.93	0.92	0.93	1009
accuracy			0.95	9999
macro avg	0.95	0.95	0.95	9999
weighted avg	0.95	0.95	0.95	9999

Thus, after applying the best parameters found from hyperparameter tuning, the model is configured as:

```
model = MLPClassifier(
    activation="relu",
    solver="adam",
    alpha=0.0001,
    hidden_layer_sizes=(50, 50, 50),
    learning_rate="constant",
    random_state=1,
    max_iter=100,
    verbose=True
)
```

3.3 Evaluation Metrics

Accuracy, Precision, Recall, and F1 score is used as evaluation metrics in experiment.

- TP = True Positive, TN = True Negative, FP = False Positive, FN = False Negative.
- Accuracy is the proportion of correctly classified instances out of the total instances
    - (TP + TN) / (TP + TN + FP + FN)
- Precision is the proportion of correctly classified positive (TP) instances out of the total instances classified as positive (TP + FP), which helps measure the accuracy of positive predictions.
    - (TP) / (TP + FP)
- Recall is the proportion of actual positive instances (TP) out of the total instances correctly classified as positive and falsely classified as negative (TP + FN)
    - (TP) / (TP + FN)
- F1 score is the harmonic mean between precision and recall.
    - 2 * (Precision * Recall) / (Precision + Recall)

3.4 Source Code

https://github.com/echung32/ics661-mlp

3.5 Training Convergence Plot

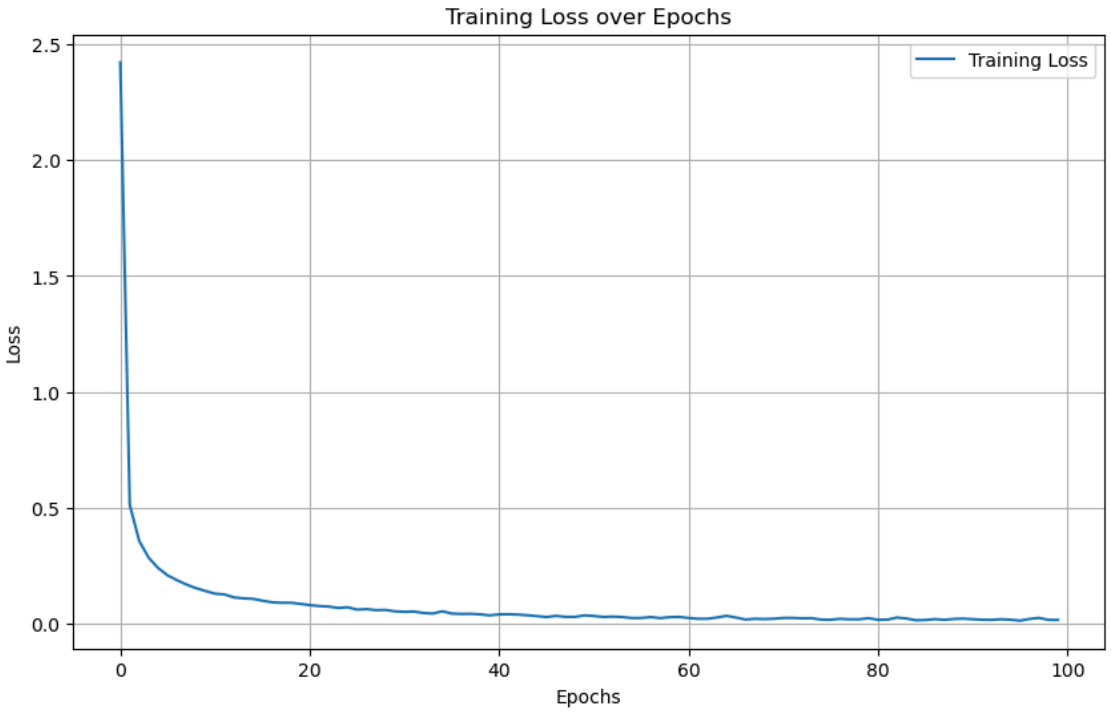

Figure 2: A chart showing the training loss minimizing over a period of 100 epochs.

3.6 Model Performance

The model was trained using the training set, then evaluated using the testing set. The scores of both models are reported below, as well as the 4 evaluation metrics:

Training Score	0.993950
Test Score	0.959296
Accuracy	0.959296
Precision	0.959656
Recall	0.959296
F1 Score	0.959234

As the Training Score (0.99) > Test Score (0.96) by 0.03 points, this may suggest that the model is slightly overfit towards the training set. This could be minimized in the future by utilizing K-fold cross-validation. Nonetheless, this difference is not statistically significant because the gap between the training and test scores (0.03 points) is relatively small.

A classification report was generated on the test set, which shows the performance of the model across multiple classes (0-9), with the individual precision, recall, f1-score. Support represents the number of

occurrences of each class within the test dataset. Compared to the first classification report found during hyperparameter tuning, the average score is higher by 0.01 (0.95 vs 0.96).

	precision	recall	f1-score	support
0	0.96	0.99	0.98	980
1	0.99	0.99	0.99	1135
2	0.97	0.96	0.96	1032
3	0.97	0.95	0.96	1010
4	0.98	0.96	0.97	982
5	0.92	0.97	0.95	892
6	0.97	0.97	0.97	958
7	0.96	0.97	0.97	1027
8	0.96	0.94	0.95	974
9	0.97	0.95	0.96	1009
accuracy			0.96	9999
macro avg	0.96	0.96	0.96	9999
weighted avg	0.97	0.96	0.96	9999

3.7 Ablation Studies

Ablation study was performed during the GridSearchCV process. Each trial variant was printed out at the end of the search with the average score, the standard deviation, and the parameters. Mean score and standard deviations are shown in the table below for every combination:

Score (Mean ± Std)	Activation	Alpha	Hidden Layer Sizes	Learning Rate	Solver
0.824 ± 0.034	tanh	0.0001	(50, 50, 50)	constant	sgd
0.731 ± 0.051	tanh	0.0001	(50, 50, 50)	constant	adam
0.891 ± 0.001	tanh	0.0001	(50, 50, 50)	adaptive	sgd
0.731 ± 0.051	tanh	0.0001	(50, 50, 50)	adaptive	adam
0.825 ± 0.035	tanh	0.0001	(50, 100, 50)	constant	sgd
0.759 ± 0.075	tanh	0.0001	(50, 100, 50)	constant	adam
0.873 ± 0.016	tanh	0.0001	(50, 100, 50)	adaptive	sgd
0.759 ± 0.075	tanh	0.0001	(50, 100, 50)	adaptive	adam
0.843 ± 0.031	tanh	0.0001	(100,)	constant	sgd
0.807 ± 0.035	tanh	0.0001	(100,)	constant	adam
0.908 ± 0.013	tanh	0.0001	(100,)	adaptive	sgd
0.807 ± 0.035	tanh	0.0001	(100,)	adaptive	adam
0.827 ± 0.030	tanh	0.05	(50, 50, 50)	constant	sgd
0.725 ± 0.059	tanh	0.05	(50, 50, 50)	constant	adam

(First 15 results are shown; the full study is available in the GitHub as *grid-search-cv-stages.csv*)